



From Training to Serving – The Deployment Gap

Bridging experiments to reliable, scalable production ML systems

What Is the Deployment Gap?

The chasm between models working in notebooks and failing under production constraints:

Environment Mismatch

Significant differences in **data, code, and infrastructure** between training and serving environments

Undefined Process

Need for repeatable, standardized steps to cross the gap successfully

Missing Requirements

Lack of proper **SLOs** (latency, availability) and governance frameworks



Training

Real time real time SLO Monitoring with the real world ops
The real world ops are the most of the real world ops.



SLO Serving

Real time real time SLO Monitoring with the real world ops
The real world ops are the most of the real world ops.



Train vs Serve: Different Worlds

Training Environment

- Throughput-oriented batch processing
- Long-running jobs tolerant to retries
- Multi-GPU hardware configurations
- Capital expenditure model: high, bursty costs

Serving Environment

- Latency/jitter-sensitive processing
- 24/7 availability with strict SLOs
- Smaller GPU/CPU footprints
- Operational expenditure: ongoing cost per request

Common Failure Modes

Understanding what breaks in the transition from lab to production:

Training–Serving Skew

Feature transformation logic differs between environments, leading to inconsistent predictions

Environment Drift

Discrepancies in libraries, CUDA versions, and drivers cause unexpected behavior

Non-Determinism

Random seeds, floating-point variations, and parallel execution create inconsistent outputs

Hidden Data Coupling

Implicit dependencies on experimental data paths that don't exist in production

Artifacts & Packaging

1 Standardize Model Format

Use [SavedModel](#) / [TorchScript](#) / [ONNX](#) to ensure compatibility across environments

2 Bundle Pre/Post-Processing

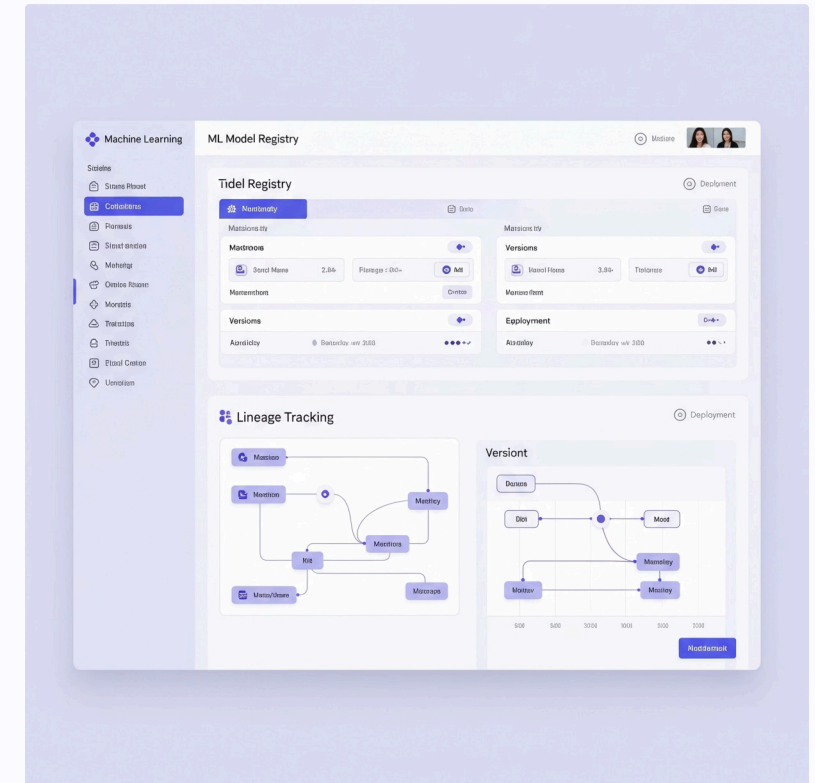
Include all transformation logic with the model to ensure consistency

3 Create Immutable Containers

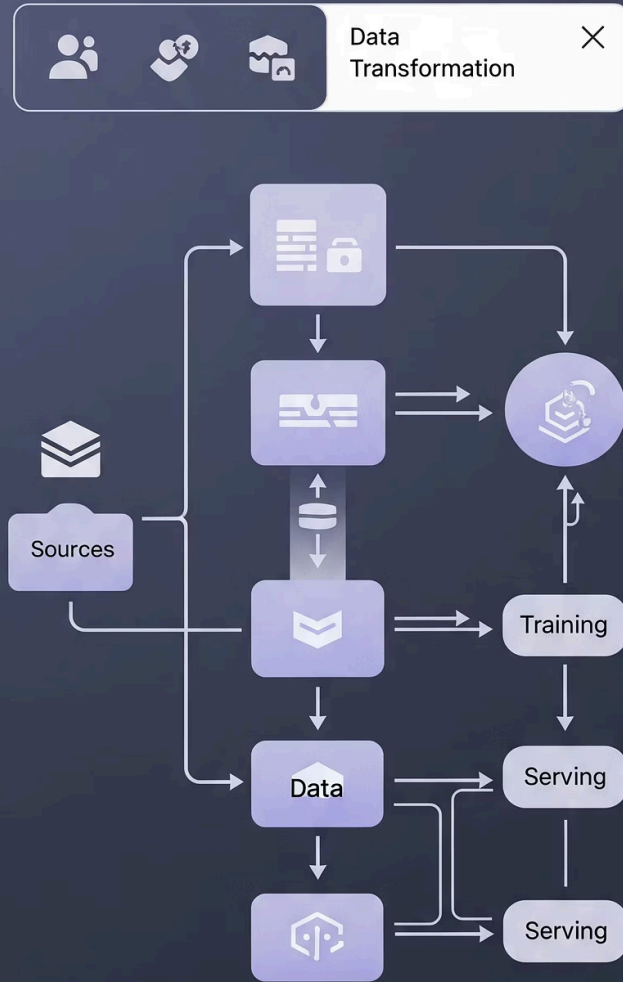
Package with Docker images and pinned dependencies to prevent drift

4 Implement Model Registry

Store artifacts with proper lineage tracking, versioning, and signatures



Feature Store Architecture



Data & Feature Parity

Feature Store



Single source of truth for both training and serving, ensuring consistency

Version & Contract



Enforce schema contracts with versioned features to prevent drift

Golden Datasets



Validate with reference data and implement automated skew detection

Vector DB Parity



Maintain consistency in embedding pipelines and retrieval mechanisms

Performance & Cost Translation



Metrics Shift

Transition from **throughput** focus to **p95/p99 latency** targets that align with user experience



Optimization Techniques

- **Dynamic batching** to amortize compute costs
- **Concurrency** tuning for optimal resource usage
- **Caching** strategies for frequent requests



Model Optimization

- **Quantization** (INT8/BF16) for faster inference
- **TensorRT** conversion for NVIDIA hardware
- **compile()** functions for framework optimization



Resource Sizing

Right-size instances to avoid GPU underutilization and excessive costs

Reliability & Scalability

Health Monitoring

Health checks (liveness/readiness) that gate traffic to prevent cascading failures

Scaling Strategy

Autoscaling with HPA + cluster autoscaler and warm pools to handle traffic spikes

Traffic Management

Load balancing with least-request algorithms and outlier ejection

High Availability

Multi-zone deployment with graceful draining and rolling updates



Testing Beyond Accuracy

Model accuracy is necessary but insufficient for production readiness:

Contract Testing

- Unit tests for feature transforms
- Input/output schema validation
- Edge case handling verification

Performance Testing

- Latency tests with hey/wrk/k6
- SLO conformance verification
- Load testing under peak conditions

Traffic Testing

- A/B testing with controlled rollout
- Shadow traffic comparison
- Production data validation

Deployment Testing

- Canary deployments
- Automated rollback triggers
- Chaos engineering scenarios



CI/CD for Models (MLOps)



End-to-End Pipeline

`data` → `train` → `eval` → `package` → `scan` → `deploy` with automated execution



Quality Gates

Enforce thresholds for accuracy, latency, cost, bias/safety before promotion



Model Promotion

Automate movement through registry stages (`dev`→`staging`→`prod`)



Deployment Strategy

Blue/green deployments with infrastructure-as-code for reproducibility

Observability & Governance

Performance Telemetry

Track [p50/p95/p99](#) latencies, error rates, resource saturation, and queue times

Distributed Tracing

Implement [OpenTelemetry](#) across entire stack: API → model → feature stores

Drift Detection

Monitor input distribution, prediction, and embedding drift with automated alerts

Compliance Artifacts

Maintain audit trails, enforce PII policies, and create [model cards](#) for transparency



Security & Compliance

Security considerations must be built in, not bolted on:



Network Security

mTLS for all connections with proper authentication and authorization on endpoints



Artifact Integrity

Signed artifacts with Software Bill of Materials (SBOM), provenance tracking, and secure secret management



Access Controls

Implement rate limits, quotas, and abuse detection systems to prevent resource exhaustion



Data Compliance

Address dataset licensing requirements and geographical data residency constraints



Anti-Patterns to Avoid

Feature Rewrite Roulette

Rewriting feature transformation code in production without comprehensive tests

The Monolithic GPU

"One big GPU box" approach that can't scale and creates a single point of failure

Hope-Driven Deployment

No rollback plan in place, leading to manual hotfixes directly in production

Hidden Cost Traps

Ignoring [egress costs](#) and [cold-start penalties](#) that can dominate your bill

Bridging Checklist (Quick Win)

Practical steps to immediately reduce your deployment gap:

✓ Unify Feature Code Path

Ensure training and serving use **identical feature transformation** code or feature store

✓ Standardize Model Artifacts

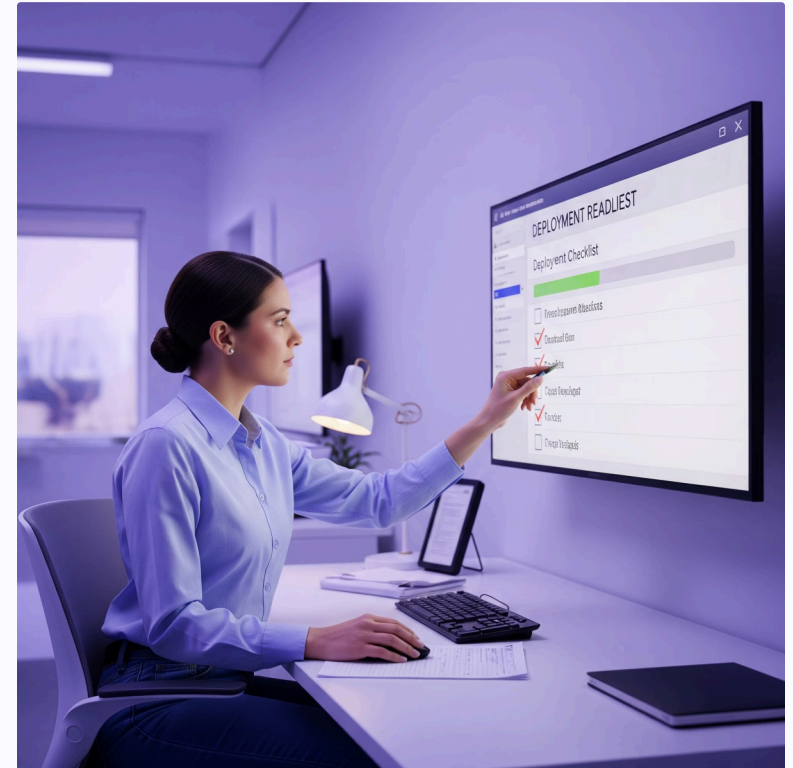
Create frozen, signed model artifacts with complete, immutable container images

✓ Define & Test SLOs

Establish clear service level objectives with comprehensive load and canary testing

✓ Prepare for Incidents

Implement monitoring, drift detection, and documented rollback procedures



Key Takeaways

Product Mindset

Treat serving infrastructure as a [product](#), not an afterthought to model development

End-to-End Alignment

Align [artifacts](#), [features](#), and [infrastructure](#) consistently across all environments

Built-In Quality

Bake in [SLOs](#), [testing](#), and [observability](#) from day one as core requirements

Disciplined Process

Close the deployment gap with structured [MLOps practices](#) and thoughtful architecture

The bridge is built through
[intentional design](#), not chance.